

Bloque 1 — Naturaleza y clasificación de las fuentes de datos

Módulo: Extracción, Transformación y Carga de Datos desde Fuentes Múltiples (5104) **Unidad**
Didáctica: UD1 — Tipología y características de las fuentes de datos **Duración estimada:** ~6 horas
Curso académico: 2026-2027

Contenidos de este bloque

Epígrafe	Contenido
1.1	Datos estructurados, semiestructurados y no estructurados en contexto ML
1.2	Clasificación por origen: SQL/NoSQL, APIs, streaming, IoT, redes sociales
1.3	Fuentes formales y no formales
1.4	Ubicación de las fuentes: local, nube pública e híbrida
1.5	Repositorios públicos de datos para ML
1.6	Metadatos y trazabilidad en proyectos de ML
1.7	Tendencias actuales en arquitecturas de datos para ML

Criterios de evaluación asociados

(CE a) Se ha determinado la ubicación de las fuentes de datos, clasificando el formato en que se encuentran almacenadas y su distribución física. (CE b) Se han clasificado las fuentes según su origen, naturaleza y formalidad. (CE c) Se han identificado las características de las fuentes no estructuradas, valorando el ecosistema de Internet como origen de información.

□ Librerías cargadas correctamente

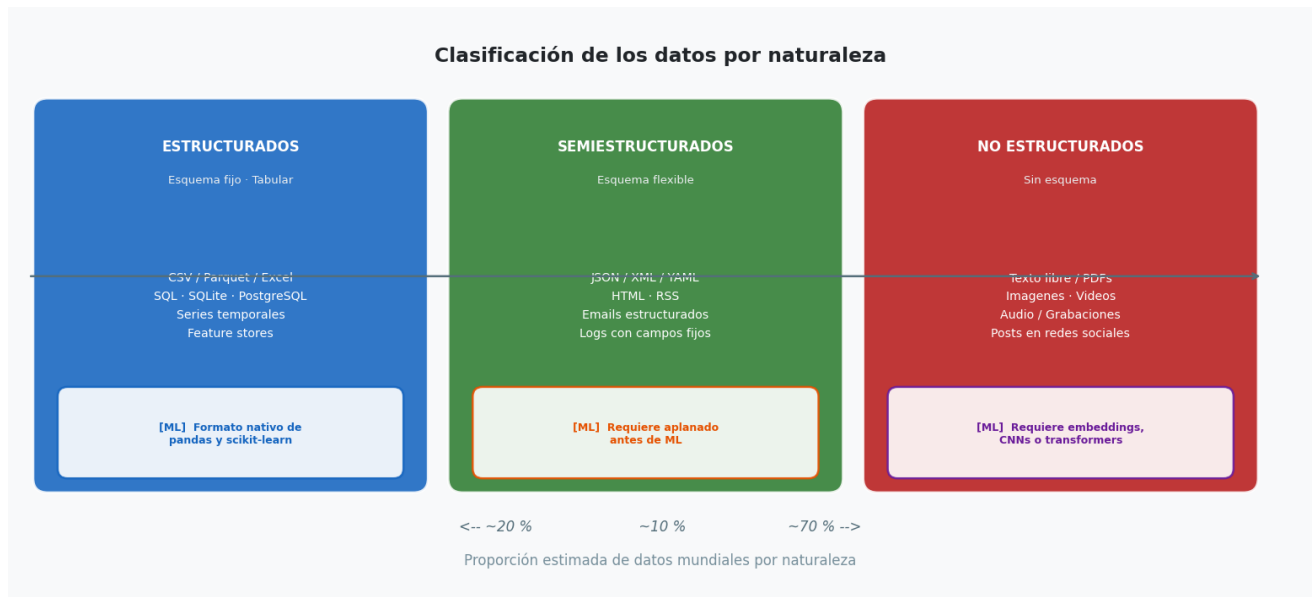
1.1 Naturaleza de los datos: estructurados, semiestructurados y no estructurados

Todo proyecto de Machine Learning comienza con una pregunta fundamental: **¿de qué tipo son mis datos?** La respuesta determina qué técnicas de extracción, qué formatos de almacenamiento y qué preprocesamiento necesitamos antes de poder entrenar cualquier modelo.

Tipo	Descripción	Ejemplos típicos	% datos mundiales (est.)
Estructurados	Organizados en filas y columnas con esquema fijo	CSV, SQL, Excel, Parquet	~20 %
Semiestructurados	Con organización flexible, sin esquema rígido	JSON, XML, HTML, YAML	~10 %

Tipo	Descripción	Ejemplos típicos	% datos mundiales (est.)
No estructurados	Sin organización predefinida	Texto, imágenes, audio, vídeo	~70 %

Dato clave: aunque el 70 % de los datos del mundo son no estructurados, los modelos clásicos de ML supervisado (regresión, árboles, SVM...) requieren datos **tabulares** (estructurados). El auge de los modelos de *deep learning* y los transformers ha permitido trabajar directamente con texto, imagen y audio, pero incluso en esos casos necesitamos pipelines ETL bien definidos.



Ejemplo — Datos estructurados con pandas

Los datos estructurados son el punto de partida más habitual en proyectos de ML. Un dataset tabular clásico tiene:

- **Filas** → observaciones (muestras de entrenamiento)
- **Columnas** → variables (features + target)

```
# Dataset de ejemplo: registros de sensores en una línea de producción industrial
datos_sensores = {
    'timestamp': pd.date_range('2026-01-01', periods=6, freq='h'),
    'temperatura': [72.3, 73.1, 71.8, 75.4, 70.2, 69.8],
    'presion_bar': [3.1, 3.2, 3.0, 3.5, 2.9, 2.8],
    'vibracion_hz': [120, 122, 119, 145, 118, 115],
    'linea': ['A', 'A', 'A', 'B', 'B', 'B'],
    'fallo': [False, False, False, True, False, False] # ← variable objetivo
}

df = pd.DataFrame(datos_sensores)
print("Dataset estructurado – sensores industriales:")
print(df.to_string(index=False))
print(f"\nDimensiones: {df.shape[0]} filas × {df.shape[1]} columnas")
print(f"Tipos:\n{df.dtypes.to_string()}")
print("\n→ Este DataFrame puede usarse directamente con scikit-learn tras un mínimo preprocesamiento")
```

Dataset estructurado – sensores industriales:

	timestamp	temperatura	presion_bar	vibracion_hz	linea	fallo
2026-01-01	00:00:00	72.3	3.1	120	A	False
2026-01-01	01:00:00	73.1	3.2	122	A	False
2026-01-01	02:00:00	71.8	3.0	119	A	False
2026-01-01	03:00:00	75.4	3.5	145	B	True
2026-01-01	04:00:00	70.2	2.9	118	B	False
2026-01-01	05:00:00	69.8	2.8	115	B	False

Dimensiones: 6 filas × 6 columnas

Tipos:

timestamp	datetime64[ns]
temperatura	float64
presion_bar	float64
vibracion_hz	int64
linea	object
fallo	bool

→ Este DataFrame puede usarse directamente con scikit-learn tras un mínimo de preprocesado.

Ejemplo — Datos semiestructurados (respuesta de API en JSON)

Una API REST devuelve habitualmente JSON. El formato es legible y flexible, pero **no directamente tabular**: puede haber campos anidados, arrays de longitud variable o claves opcionales.

```
# Simulamos la respuesta JSON de una API de telemetría de vehículos eléctricos
respuesta_api = '''
{
  "vehiculos": [
    {
      "id": "EV-001",
      "bateria": {"carga_pct": 87, "temperatura_c": 28, "ciclos": 312},
      "gps": {"lat": 37.386, "lon": -5.992},
      "alertas": ["bateria_caliente"]
    },
    {
      "id": "EV-002",
      "bateria": {"carga_pct": 42, "temperatura_c": 23, "ciclos": 891},
      "gps": {"lat": 37.401, "lon": -5.980},
      "alertas": []
    },
    {
      "id": "EV-003",
      "bateria": {"carga_pct": 95, "temperatura_c": 25, "ciclos": 54},
      "gps": {"lat": 37.392, "lon": -5.995},
      "alertas": ["bateria_caliente", "revision_pendiente"]
    }
  ]
}
'''

datos = json.loads(respuesta_api)

# Intento de aplanado con json_normalize
df_raw = pd.json_normalize(datos['vehiculos'])
print("Resultado de json_normalize() – sin tratar:")
print(df_raw.to_string())
```

```

print("\n⚠ Problema: 'alertas' contiene listas de longitud variable → no es tabular")
print("    Solución: multi-hot encoding o explosión de filas según el caso de uso.")

# Aplanado útil para ML: extraemos features escalares
registros = []
for v in datos['vehiculos']:
    registros.append({
        'id': v['id'],
        'carga_pct': v['bateria']['carga_pct'],
        'temp_bateria': v['bateria']['temperatura_c'],
        'ciclos': v['bateria']['ciclos'],
        'lat': v['gps']['lat'],
        'lon': v['gps']['lon'],
        'n_alertas': len(v['alertas']), # feature numérica derivada
        'alerta_calor': int('bateria_caliente' in v['alertas']) # one-hot manual
    })

df_ml = pd.DataFrame(registros)
print("\nDataFrame listo para ML tras aplanado:")
print(df_ml.to_string(index=False))

```

Resultado de json_normalize() – sin tratar:

	id	alertas	bateria.carga_pct
bateria.temperatura_c	bateria.ciclos	gps.lat	gps.lon
0	EV-001	[bateria_caliente]	
87		28	312 37.386 -5.992
1	EV-002	[]	
42		23	891 37.401 -5.980
2	EV-003	[bateria_caliente, revision_pendiente]	
95		25	54 37.392 -5.995

⚠ Problema: 'alertas' contiene listas de longitud variable → no es tabular directamente.

Solución: multi-hot encoding o explosión de filas según el caso de uso.

DataFrame listo para ML tras aplanado:

	id	carga_pct	temp_bateria	ciclos	lat	lon	n_alertas	alerta_calor
EV-001		87	28	312	37.386	-5.992	1	1
EV-002		42	23	891	37.401	-5.980	0	0
EV-003		95	25	54	37.392	-5.995	2	1

Ejemplo — Datos no estructurados (texto libre)

El texto libre no tiene esquema: no hay columnas ni tipos predefinidos. Para usarlo en ML necesitamos técnicas como **tokenización**, **embeddings** o **TF-IDF** que lo transformen en representaciones numéricas.

```

# Simulamos reseñas de productos (texto no estructurado)
resenas = [
    "El producto llegó a tiempo y en perfectas condiciones. La calidad es excelente",
    "Decepcionante. El embalaje estaba dañado y el artículo no funciona bien.",
    "Precio razonable para lo que ofrece. Funciona, pero nada del otro mundo.",
    "¡Increíble! Superó todas mis expectativas. Lo recomiendo sin dudarlo.",
    "Tardó mucho en llegar. El producto es correcto pero el servicio mejorable."
]

etiquetas = [1, 0, 1, 1, 0] # 1 = positiva, 0 = negativa ← ¡estas las pone un hu

print("Datos no estructurados – reseñas de productos:")
print("-" * 60)

```

```

for i, (r, e) in enumerate(zip(resenas, etiquetas)):
    sentimiento = "😊 Positiva" if e == 1 else "😞 Negativa"
    print(f"[{sentimiento}] {r}")

print()
print("Para convertirlos en features numéricas podemos usar:")
print(" • TF-IDF vectorization (sklearn.feature_extraction.text.TfidfVectorizer)")
print(" • Word embeddings (Word2Vec, GloVe, FastText)")
print(" • Sentence embeddings (sentence-transformers, OpenAI embeddings)")
print(" • LLM fine-tuning (BERT, RoBERTa, LLaMA...)" )
print()
print("La cadena de procesado es:")
print(" Texto en bruto → tokenización → vectorización → modelo ML/DL → predicción")

```

Datos no estructurados – reseñas de productos:

```

-----
[😊 Positiva] El producto llegó a tiempo y en perfectas condiciones. La calidad es
excelente.
[😞 Negativa] Decepcionante. El embalaje estaba dañado y el artículo no funciona
bien.
[😊 Positiva] Precio razonable para lo que ofrece. Funciona, pero nada del otro
mundo.
[😊 Positiva] ¡Increíble! Superó todas mis expectativas. Lo recomiendo sin dudarlo.
[😞 Negativa] Tardó mucho en llegar. El producto es correcto pero el servicio
mejorable.

```

Para convertirlos en features numéricas podemos usar:

- TF-IDF vectorization (sklearn.feature_extraction.text.TfidfVectorizer)
- Word embeddings (Word2Vec, GloVe, FastText)
- Sentence embeddings (sentence-transformers, OpenAI embeddings)
- LLM fine-tuning (BERT, RoBERTa, LLaMA...)

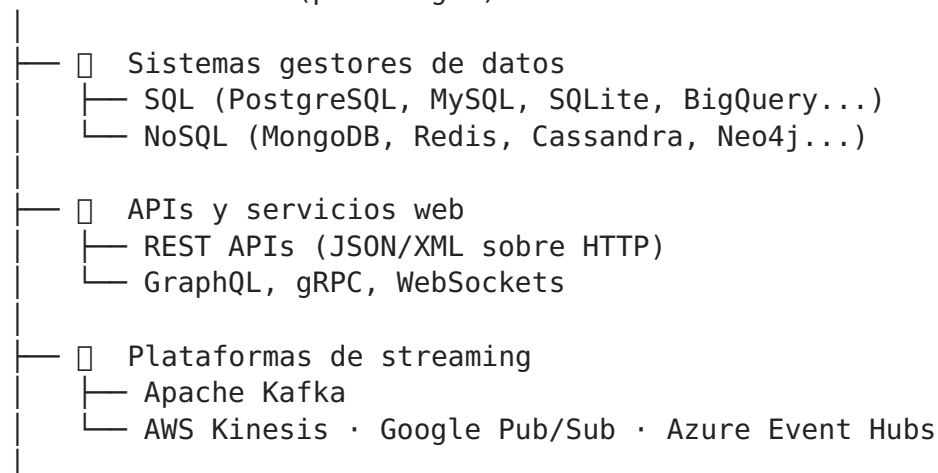
La cadena de procesado es:

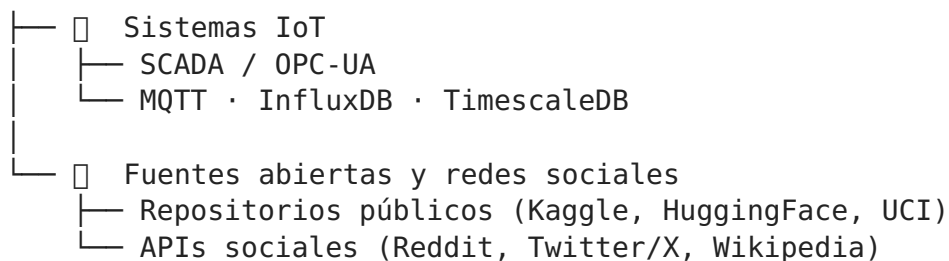
Texto en bruto → tokenización → vectorización → modelo ML/DL → predicción

1.2 Clasificación de las fuentes de datos por origen

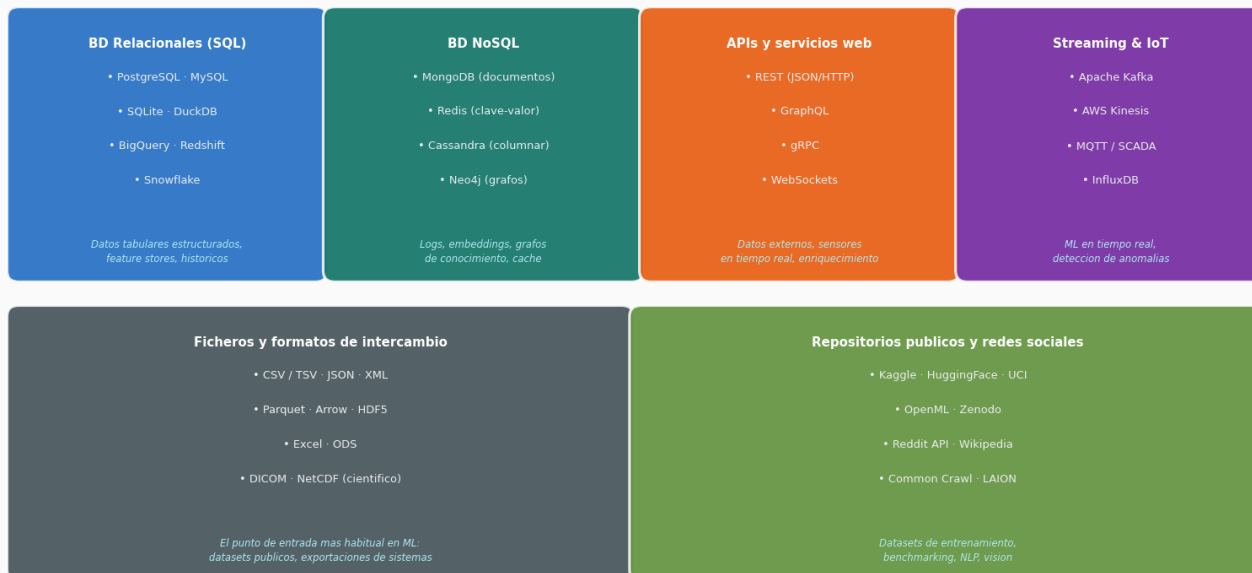
Más allá de la naturaleza de los datos, en un proyecto real de ML necesitamos saber **de dónde vienen**. Cada tipo de origen tiene sus propias herramientas de conexión, sus latencias y sus implicaciones de gobernanza.

FUENTES DE DATOS (por origen)





Fuentes de datos en proyectos de ML — clasificación por origen



Ejemplo — Consulta SQL como fuente de datos para ML

Las bases de datos relacionales son la fuente más común en entornos corporativos. En ML se usan para extraer el **conjunto de entrenamiento** a partir de datos transaccionales históricos.

```
# Simulamos una base de datos con SQLite (no requiere servidor)
conn = sqlite3.connect(':memory:') # BD en memoria

# Creamos una tabla de transacciones de e-commerce
conn.executescript("""
CREATE TABLE transacciones (
    id            INTEGER PRIMARY KEY,
    cliente_id    INTEGER,
    importe       REAL,
    n_items       INTEGER,
    canal         TEXT,           -- 'web', 'app', 'tienda'
    hora          INTEGER,       -- hora del día (0-23)
    es_fraude     INTEGER        -- 0 = legítima, 1 = fraudulenta ← TARGET
);

INSERT INTO transacciones VALUES
(1, 101, 45.80, 2, 'web', 14, 0),
(2, 202, 892.50, 1, 'app', 3, 1),
(3, 101, 23.10, 3, 'web', 10, 0),
(4, 303, 12.00, 1, 'tienda', 11, 0),
(5, 404, 750.00, 1, 'app', 2, 1),
(6, 101, 67.30, 4, 'web', 16, 0),
(7, 505, 310.00, 2, 'web', 1, 1);
```

```

"""
)

# Consulta SQL para construir el dataset de ML
query = """
SELECT
    importe,
    n_items,
    canal,
    hora,
    CASE WHEN hora BETWEEN 0 AND 5 THEN 1 ELSE 0 END AS hora_nocturna,
    es_fraude
FROM transacciones
WHERE importe > 0
ORDER BY id
"""

df_fraude = pd.read_sql_query(query, conn)
print("Dataset extraído desde SQL para detección de fraude:")
print(df_fraude.to_string(index=False))
print(f"\nRegistros: {len(df_fraude)} | Fraudes: {df_fraude['es_fraude'].sum()} ({c
print("\n→ Con pd.read_sql_query() conectamos directamente cualquier BD SQL a un Da
conn.close()

```

Dataset extraído desde SQL para detección de fraude:

importe	n_items	canal	hora	hora_nocturna	es_fraude
45.8	2	web	14	0	0
892.5	1	app	3	1	1
23.1	3	web	10	0	0
12.0	1	tienda	11	0	0
750.0	1	app	2	1	1
67.3	4	web	16	0	0
310.0	2	web	1	1	1

Registros: 7 | Fraudes: 3 (43%)

→ Con `pd.read_sql_query()` conectamos directamente cualquier BD SQL a un DataFrame.

Ejemplo — API REST como fuente de datos

Las APIs REST son una fuente clave para obtener datos externos: clima, noticias, precios financieros, datos geoespaciales... La librería `requests` es el estándar en Python.

□ Usamos [Open-Meteo](#) — una API meteorológica **gratuita y sin registro** — para obtener datos reales de temperatura en Sevilla. Estos datos podrían alimentar un modelo de predicción de consumo energético.

```

import requests

# API pública de meteorología – sin clave API requerida
url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 37.39,      # Sevilla
    "longitude": -5.99,
    "hourly": "temperature_2m,precipitation,windspeed_10m",
    "forecast_days": 1,
    "timezone": "Europe/Madrid"
}

```

```

try:
    response = requests.get(url, params=params, timeout=10)
    response.raise_for_status()
    datos = response.json()

    df_clima = pd.DataFrame({
        'hora':      datos['hourly']['time'],
        'temp_c':    datos['hourly']['temperature_2m'],
        'precip_mm': datos['hourly']['precipitation'],
        'viento_kmh': datos['hourly']['windspeed_10m']
    })
    df_clima['hora'] = pd.to_datetime(df_clima['hora'])

    print("Datos meteorológicos obtenidos desde la API Open-Meteo (Sevilla, hoy):")
    print(df_clima.head(12).to_string(index=False))
    print(f"\nTotal de registros horarios: {len(df_clima)}")
    print(f"Temperatura media: {df_clima['temp_c'].mean():.1f} °C")
    print(f"Temperatura máxima: {df_clima['temp_c'].max():.1f} °C")
    print("\n→ Este DataFrame ya puede usarse para entrenar un modelo de predicción")

except requests.exceptions.RequestException as e:
    print(f"No se pudo conectar a la API (sin conexión a internet en el entorno): {e}")
    print("\nSimulamos datos para continuar el ejemplo:")
    df_clima = pd.DataFrame({
        'hora':      pd.date_range('2026-01-01', periods=12, freq='h'),
        'temp_c':    np.random.normal(18, 4, 12).round(1),
        'precip_mm': np.zeros(12),
        'viento_kmh': np.random.uniform(5, 30, 12).round(1)
    })
    print(df_clima.to_string(index=False))

```

No se pudo conectar a la API (sin conexión a internet en el entorno):
 HTTPConnectionPool(host='api.open-meteo.com', port=443): Max retries exceeded with url: /v1/forecast?latitude=37.39&longitude=-5.99&hourly=temperature_2m%2Cprecipitation%2Cwindspeed_10m&forecast_days=1&timezone=Europe%2FMadrid (Caused by ProxyError('Unable to connect to proxy', OSError('Tunnel connection failed: 403 Forbidden')))

Simulamos datos para continuar el ejemplo:

	hora	temp_c	precip_mm	viento_kmh
2026-01-01	00:00:00	24.5	0.0	10.8
2026-01-01	01:00:00	21.3	0.0	17.0
2026-01-01	02:00:00	16.8	0.0	13.5
2026-01-01	03:00:00	21.6	0.0	22.1
2026-01-01	04:00:00	12.5	0.0	24.4
2026-01-01	05:00:00	12.8	0.0	29.0
2026-01-01	06:00:00	21.4	0.0	27.3
2026-01-01	07:00:00	8.8	0.0	20.7
2026-01-01	08:00:00	20.8	0.0	24.8
2026-01-01	09:00:00	22.7	0.0	23.8
2026-01-01	10:00:00	19.7	0.0	13.9
2026-01-01	11:00:00	14.1	0.0	22.2

Ejemplo — Datos de streaming (simulación)

En **ML en tiempo real** (online learning, detección de anomalías, recomendaciones instantáneas), los datos llegan como un flujo continuo. Plataformas como **Apache Kafka** o **AWS Kinesis** gestionan estos flujos. Aquí simulamos cómo se recibirían y procesarían.

```

import time
import random
from collections import deque

def simular_sensor_iot(n_muestras=10, intervalo_s=0):
    """Generador que simula un sensor de temperatura industrial enviando lecturas."""
    for i in range(n_muestras):
        yield {
            'ts': pd.Timestamp.now(),
            'sensor_id': random.choice(['S-01', 'S-02', 'S-03']),
            'temp_c': round(random.gauss(70, 3), 2), # media 70 °C, sd 3
            'presion': round(random.gauss(3.0, 0.2), 3),
            'anomia': random.random() < 0.1 # 10% de probabilidad
        }
        time.sleep(intervalo_s) # intervalo_s=0 para no esperar en el cuaderno

print("Simulación de stream de datos IoT (10 mensajes):")
print("-" * 65)

# Ventana deslizante para calcular la media móvil (patrón común en streaming ML)
ventana = deque(maxlen=5)
registros = []

for mensaje in simular_sensor_iot(n_muestras=10):
    ventana.append(mensaje['temp_c'])
    media_movil = round(sum(ventana) / len(ventana), 2)
    alerta = "⚠ ANOMALIA" if mensaje['anomia'] else ""

    print(f" [{mensaje['sensor_id']}] temp={mensaje['temp_c']:5.2f}°C  "
          f"presion={mensaje['presion']:.3f}bar  "
          f"media_mov(5)={media_movil:5.2f}°C {alerta}")
    registros.append(**mensaje, 'media_movil_5': media_movil)

df_stream = pd.DataFrame(registros)
print(f"\n→ En un pipeline real, cada mensaje se procesaría con Kafka + Faust o Spark")
print(f" El modelo de detección de anomalías recibiría features en tiempo real y

```

Simulación de stream de datos IoT (10 mensajes):

```

-----
[S-03] temp=63.17°C  presion=2.648bar  media_mov(5)=63.17°C
[S-02] temp=69.15°C  presion=3.143bar  media_mov(5)=66.16°C
[S-02] temp=69.41°C  presion=3.505bar  media_mov(5)=67.24°C
[S-03] temp=71.55°C  presion=3.177bar  media_mov(5)=68.32°C
[S-03] temp=72.65°C  presion=3.077bar  media_mov(5)=69.19°C
[S-01] temp=66.72°C  presion=3.066bar  media_mov(5)=69.90°C
[S-02] temp=72.72°C  presion=3.247bar  media_mov(5)=70.61°C
[S-02] temp=71.97°C  presion=2.839bar  media_mov(5)=71.12°C
[S-01] temp=69.48°C  presion=3.010bar  media_mov(5)=70.71°C
[S-01] temp=65.76°C  presion=2.661bar  media_mov(5)=69.33°C

```

→ En un pipeline real, cada mensaje se procesaría con Kafka + Faust o Spark Structured Streaming.

El modelo de detección de anomalías recibiría features en tiempo real y devolvería predicciones.

1.3 Fuentes formales y no formales

Una dimensión frecuentemente olvidada en los proyectos ML es si los datos proceden de **fuentes formales** (gestionadas, validadas, con gobernanza) o **no formales** (brutas, sin curación). Esta distinción tiene un impacto directo en la calidad del modelo resultante.

Dimensión	Fuentes formales	Fuentes no formales
Origen	Bases de datos corporativas, ERPs, CRMs, sistemas transaccionales	Redes sociales, foros, emails, PDFs escaneados, audios
Estructura	Esquema definido y documentado	Sin esquema o muy variable
Calidad	Validada, con controles de integridad	Variable, ruido elevado
Etiquetado	Frecuentemente ya etiquetado (ej: historial de fraudes)	Requiere anotación humana (costosa)
Acceso	Controlado, API interna o SQL	Scraping, APIs públicas, descarga manual
Privacidad	GDPR/LOPDGDD estricto	Datos "aparentemente" públicos, con implicaciones legales
Ejemplo ML	Dataset de clientes para churn prediction	Comentarios de Twitter para análisis de sentimiento

⚠ **Atención legal:** los datos de redes sociales son "públicos" pero su uso para entrenar modelos puede estar sujeto a los **Términos de Servicio** de la plataforma y al **Reglamento Europeo de Inteligencia Artificial (AI Act)**. Siempre hay que verificar la licencia y el propósito permitido.

Implicaciones para el pipeline ETL

- Las fuentes **formales** permiten pipelines automatizados y reproducibles.
- Las fuentes **no formales** requieren etapas adicionales: limpieza de ruido, detección de idioma, anonimización, etc.
- En la práctica, los mejores modelos combinan ambas: datos formales para el núcleo y datos no formales para enriquecimiento.

```
# — Comparativa visual: calidad típica de datos formales vs no formales —
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
fig.patch.set_facecolor('#FAFAFA')
fig.suptitle('Comparativa de características: fuentes formales vs no formales',
             fontsize=12, fontweight='bold', color='#212529')

dimensiones = ['Compleitud\n(sin nulos)', 'Consistencia\n(sin errores)',
               'Etiquetado', 'Accesibilidad\n(API/SQL)', 'Volumen\n(potencial)']

formales      = [0.92, 0.88, 0.80, 0.90, 0.45]
no_formales   = [0.55, 0.48, 0.25, 0.60, 0.95]

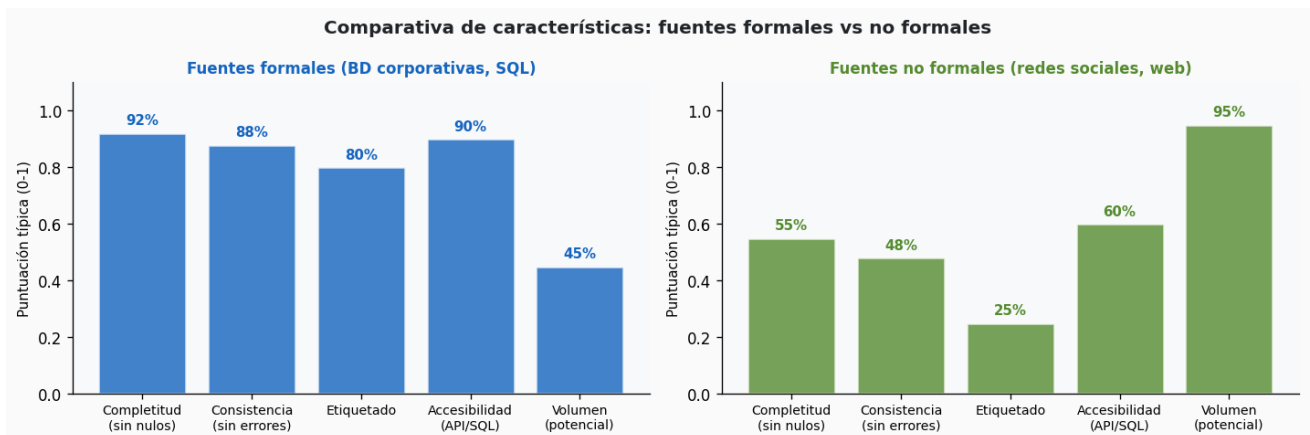
x = np.arange(len(dimensiones))
width = 0.35

ax1, ax2 = axes
for ax, vals, color, titulo in [
    (ax1, formales, '#1565C0', 'Fuentes formales (BD corporativas, SQL)'),
```

```

(ax2, no_formales, '#558B2F', 'Fuentes no formales (redes sociales, web)')
]:
bars = ax.bar(x, vals, color=color, alpha=0.8, edgecolor='white', linewidth=1.5)
ax.set_ylim(0, 1.1)
ax.set_xticks(x)
ax.set_xticklabels(dimensiones, fontsize=8.5)
ax.set_ylabel('Puntuación típica (0-1)', fontsize=9)
ax.set_title(titulo, fontsize=10, fontweight='bold', color=color)
ax.set_facecolor('#F8F9FA')
ax.spines[['top', 'right']].set_visible(False)
for bar, v in zip(bars, vals):
    ax.text(bar.get_x() + bar.get_width()/2, v + 0.02, f'{v:.0%}',
            ha='center', va='bottom', fontsize=9, fontweight='bold', color=col
plt.tight_layout()
plt.show()
print("\n→ Las fuentes no formales destacan en volumen (internet es enorme) pero re
print("    mucho más trabajo de preprocesado y etiquetado para ser útiles en ML.")

```



→ Las fuentes no formales destacan en volumen (internet es enorme) pero requieren mucho más trabajo de preprocesado y etiquetado para ser útiles en ML.

1.4 Ubicación de las fuentes: local, nube pública e híbrida

Saber **dónde están** los datos es tan importante como saber **qué son**. La ubicación afecta a:

- **Latencia** de acceso (¿cuánto tarda en llegar el dato?)
- **Coste** (transferencia, almacenamiento, cómputo)
- **Seguridad y privacidad** (datos sensibles en local, GDPR)
- **Escalabilidad** (¿puede crecer con el volumen de datos?)
- **Reproducibilidad** del pipeline ML

Tres paradigmas de ubicación

Paradigma	Descripción	Ventajas	Inconvenientes
Local (on-premise)	Datos en servidores propios o portátil del data scientist	Control total, sin latencia de red, sin coste de transferencia	Escalabilidad limitada, mantenimiento costoso
Nube pública	Datos en AWS, GCP o Azure	Escalabilidad infinita, servicios gestionados, colaboración fácil	Coste variable, dependencia del proveedor (vendor lock-in)

Paradigma	Descripción	Ventajas	Inconvenientes
Híbrido / Multi-cloud	Combinación de local y nube	Equilibrio entre control y escalabilidad	Mayor complejidad de gestión y seguridad

Tendencia 2024-2026: el modelo **híbrido** domina en enterprise. Los datos sensibles (PII, datos de salud) permanecen en local; el entrenamiento pesado y el almacenamiento masivo van a la nube. El concepto de **data lakehouse** (Delta Lake, Apache Iceberg) unifica ambos mundos.

Principales servicios cloud para datos ML — comparativa por proveedor

AWS (Amazon)		GCP (Google)		Azure (Microsoft)	
Almacenamiento	S3 — Almacenamiento objetos	Almacenamiento	Cloud Storage (GCS)	Almacenamiento	Azure Blob Storage
SQL	RDS / Aurora — SQL gestionado	SQL	BigQuery — Data warehouse SQL	SQL	Azure SQL / CosmosDB
NoSQL	DynamoDB — NoSQL serverless	NoSQL	Firestore — NoSQL documentos	NoSQL	Azure Data Lake Gen2
ETL	Glue — ETL serverless	ETL	Dataflow — ETL / streaming	ETL	Data Factory — ETL
ML Platform	SageMaker — ML end-to-end	ML Platform	Vertex AI — ML end-to-end	ML Platform	Azure ML — ML end-to-end
Streaming	Kinesis — Streaming datos	Streaming	Pub/Sub — Messaging	Streaming	Event Hubs — Streaming

```
# — Ejemplo: acceso a datos según ubicación con Python —————

print("=" * 60)
print("PATRON DE ACCESO SEGUN UBICACION")
print("=" * 60)

print("""
[LOCAL] Fichero en disco
_____

import pandas as pd
df = pd.read_parquet('datos/sensores_2026.parquet')

[AWS S3]
_____

import boto3, pandas as pd
s3 = boto3.client('s3')
obj = s3.get_object(Bucket='mi-bucket', Key='datos/sensores_2026.parquet')
df = pd.read_parquet(obj['Body'])

[Google Cloud Storage]
_____

import pandas as pd
df = pd.read_parquet('gs://mi-bucket/datos/sensores_2026.parquet')

[Azure Blob Storage]
_____

from azure.storage.blob import BlobServiceClient
import pandas as pd, io
client = BlobServiceClient.from_connection_string(conn_str)
blob = client.get_blob_client('mi-contenedor', 'datos/sensores_2026.parquet')
df = pd.read_parquet(io.BytesIO(blob.download_blob().readall()))
```

→ En todos los casos el resultado es el mismo: un DataFrame de pandas.
La diferencia está en la autenticación y en la latencia de lectura.
""")

=====

PATRON DE ACCESO SEGUN UBICACION

=====

[LOCAL] Fichero en disco

```
import pandas as pd
df = pd.read_parquet('datos/sensores_2026.parquet')
```

[AWS S3]

```
import boto3, pandas as pd
s3 = boto3.client('s3')
obj = s3.get_object(Bucket='mi-bucket', Key='datos/sensores_2026.parquet')
df = pd.read_parquet(obj['Body'])
```

[Google Cloud Storage]

```
import pandas as pd
df = pd.read_parquet('gs://mi-bucket/datos/sensores_2026.parquet')
```

[Azure Blob Storage]

```
from azure.storage.blob import BlobServiceClient
import pandas as pd, io
client = BlobServiceClient.from_connection_string(conn_str)
blob = client.get_blob_client('mi-contenedor', 'datos/sensores_2026.parquet')
df = pd.read_parquet(io.BytesIO(blob.download_blob().readall()))
```

→ En todos los casos el resultado es el mismo: un DataFrame de pandas.
La diferencia está en la autenticación y en la latencia de lectura.

1.5 Repositorios públicos de datos para ML

Para prototipar, enseñar y hacer benchmarking no es necesario tener datos propios. Existe un ecosistema rico de datasets públicos, algunos con millones de muestras, distribuidos bajo licencias abiertas.

Repositorio	URL	Especialidad	Acceso Python
Kaggle	kaggle.com	Competiciones, tabular, NLP, CV	kaggle CLI / API
Hugging Face Datasets	huggingface.co/datasets	NLP, audio, CV, multimodal	datasets library
UCI ML Repository	archive.ics.uci.edu	Clásicos tabulares	ucimlrepo / descarga
OpenML	openml.org	Benchmarking, tabular	openml library
Zenodo	zenodo.org	Científico, multidisciplinar	Descarga directa / API

Repositorio	URL	Especialidad	Acceso Python
Google Dataset Search	datasetsearch.research.google.com	Metabuscador	Web
Papers With Code	paperswithcode.com/datasets	Datasets ligados a papers ML	Web / API

```
# — Datasets clásicos disponibles directamente en scikit-learn —
from sklearn import datasets

catalogo = {
    'iris': ('clasificación', 150, 4, 3, 'flores por especie'),
    'wine': ('clasificación', 178, 13, 3, 'vino por variedad'),
    'breast_cancer': ('clasificación', 569, 30, 2, 'tumor maligno/benigno'),
    'diabetes': ('regresión', 442, 10, '-', 'progresión de la diabetes'),
    'california': ('regresión', 20640, 8, '-', 'precio de viviendas en Calif')
}

print("Datasets incluidos en scikit-learn (sin descarga):")
print(f"{'Nombre':<18} {'Tarea':<15} {'Muestras':>9} {'Features':>9} {'Clases':>7}")
print("-" * 80)
for nombre, (tarea, n, f, c, desc) in catalogo.items():
    print(f"{'nombre':<18} {'tarea':<15} {'n':>9,} {'f':>9} {'str(c):>7} {'desc:':>7}")

# Cargamos California housing como ejemplo práctico
from sklearn.datasets import fetch_california_housing
from sklearn.datasets import load_iris as _load_iris_tmp
try:
    housing = fetch_california_housing(as_frame=True)
    df_housing = housing.frame
except Exception:
    # Sin conexion a internet: simulamos el dataset
    import numpy as np
    rng2 = np.random.default_rng(1)
    n2 = 500
    df_housing = __import__('pandas').DataFrame({
        'MedInc': rng2.uniform(1, 15, n2).round(4),
        'HouseAge': rng2.uniform(1, 52, n2).round(1),
        'AveRooms': rng2.uniform(2, 10, n2).round(2),
        'AveBedrms': rng2.uniform(0.5, 3, n2).round(2),
        'Population': rng2.integers(3, 3500, n2).astype(float),
        'AveOccup': rng2.uniform(1, 6, n2).round(2),
        'Latitude': rng2.uniform(32, 42, n2).round(4),
        'Longitude': rng2.uniform(-124, -114, n2).round(4),
        'MedHouseVal': rng2.uniform(0.15, 5, n2).round(4),
    })
    print("(Datos simulados – sin acceso a internet)")
print(f"\nDataset California Housing:")
print(df_housing.describe().round(2))
print(f"\nTarget: precio medio de vivienda en cientos de miles de dólares")
print(f"  Min: ${df_housing['MedHouseVal'].min():.2f}x100k  "
      f"Max: ${df_housing['MedHouseVal'].max():.2f}x100k  "
      f"Media: ${df_housing['MedHouseVal'].mean():.2f}x100k")
```

Datasets incluidos en scikit-learn (sin descarga):

Nombre	Tarea	Muestras	Features	Clases	Descripción
iris	clasificación	150	4	3	flores por especie
wine	clasificación	178	13	3	vino por variedad
breast_cancer benigno	clasificación	569	30	2	tumor maligno/
diabetes	regresión	442	10	-	progresión de la
diabetes california	regresión	20,640	8	-	precio de viviendas en California

```
/sessions/loving-practical-knuth/.local/lib/python3.10/site-packages/sklearn/  
datasets/_base.py:1518: UserWarning: Retry downloading from url: https://  
ndownloader.figshare.com/files/5976036
```

```
warnings.warn(f"Retry downloading from url: {remote.url}")
```

(Datos simulados – sin acceso a internet)

Dataset California Housing:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
count	500.00	500.00	500.00	500.00	500.00	500.00	500.00	
mean	7.88	27.21	6.06	1.73	1688.60	3.47	36.83	
std	4.02	14.95	2.29	0.71	1036.66	1.44	3.00	
min	1.03	1.20	2.00	0.51	16.00	1.00	32.00	
25%	4.61	14.50	4.00	1.09	761.50	2.27	34.01	
50%	7.69	26.60	6.12	1.74	1696.00	3.40	36.70	
75%	11.51	41.12	8.00	2.29	2575.75	4.77	39.44	
max	14.99	52.00	10.00	2.99	3488.00	5.98	41.92	

	Longitude	MedHouseVal
count	500.00	500.00
mean	-118.82	2.56
std	2.91	1.42
min	-123.97	0.17
25%	-121.25	1.31
50%	-118.64	2.61
75%	-116.36	3.76
max	-114.00	4.99

Target: precio medio de vivienda en cientos de miles de dólares

Min: \$0.17x100k Max: \$4.99x100k Media: \$2.56x100k

```
# — Acceso a HuggingFace Datasets (requiere: pip install datasets) ————  
print("Ejemplo de acceso a HuggingFace Datasets:")  
print("""from datasets import load_dataset  
  
# Dataset IMDB (50.000 reseñas de películas, sentimiento binario)  
dataset = load_dataset('imdb')  
print(dataset)  
# DatasetDict({  
#   train: Dataset({features: ['text', 'label'], num_rows: 25000}),  
#   test: Dataset({features: ['text', 'label'], num_rows: 25000})  
# })  
  
# Convertir a pandas  
df_train = dataset['train'].to_pandas()  
print(df_train.head())  
  
# Dataset de imágenes: MNIST  
mnist = load_dataset('mnist')  
""")
```

```

print("\nAlgunos de los datasets más populares en HuggingFace (agosto 2026):")
populares = [
    ('imdb', 'NLP', '50k reseñas', 'Análisis de sentimiento'),
    ('squad', 'NLP', '100k preguntas', 'Question Answering'),
    ('imagenet-1k', 'Visión', '1.2M imágenes', 'Clasificación imágenes'),
    ('common_voice_13_0', 'Audio', '100+ idiomas', 'Reconocimiento voz (ASR)'),
    ('glue', 'NLP', 'Múltiples', 'Benchmark NLP multi-tarea'),
    ('ms_marco', 'NLP', '1M consultas', 'Information Retrieval / RAG')
]
print(f"{'Nombre':<25} {'Tipo':<10} {'Tamaño':<15} {'Uso típico'}")
print("-" * 70)
for nombre, tipo, tam, uso in populares:
    print(f"{'Nombre':<25} {'Tipo':<10} {'Tamaño':<15} {'Uso'}")

```

Ejemplo de acceso a HuggingFace Datasets:

```
from datasets import load_dataset
```

```
# Dataset IMDB (50.000 reseñas de películas, sentimiento binario)
```

```
dataset = load_dataset('imdb')
```

```
print(dataset)
```

```
# DatasetDict({
```

```
#   train: Dataset({features: ['text', 'label'], num_rows: 25000}),
```

```
#   test: Dataset({features: ['text', 'label'], num_rows: 25000})
```

```
# })
```

```
# Convertir a pandas
```

```
df_train = dataset['train'].to_pandas()
```

```
print(df_train.head())
```

```
# Dataset de imágenes: MNIST
```

```
mnist = load_dataset('mnist')
```

Algunos de los datasets más populares en HuggingFace (agosto 2026):

Nombre	Tipo	Tamaño	Uso típico
imdb	NLP	50k reseñas	Análisis de sentimiento
squad	NLP	100k preguntas	Question Answering
imagenet-1k	Visión	1.2M imágenes	Clasificación imágenes
common_voice_13_0	Audio	100+ idiomas	Reconocimiento voz (ASR)
glue	NLP	Múltiples	Benchmark NLP multi-tarea
ms_marco	NLP	1M consultas	Information Retrieval / RAG

1.6 Metadatos y trazabilidad en proyectos de ML

Los **metadatos** son datos sobre los datos: describen qué contiene un dataset, cómo fue generado, cuándo, por quién y con qué licencia. En ML son especialmente críticos porque permiten **reproducir** experimentos y **auditar** modelos.

Tipos de metadatos

METADATOS

└─ Descriptivos → nombre, descripción, tags, dominio de aplicación

- └ Estructurales → esquema (tipos de columnas), forma (filas × cols), versión
- └ Administrativos → autor, fecha de creación/modificación, licencia, fuente
- └ De calidad → % valores nulos por columna, distribución de clases, rango de valores

Data Cards y Model Cards

La industria ha adoptado el concepto de **Data Card** (Google) y **Model Card** (Hugging Face) — documentos estructurados que acompañan a cada dataset o modelo y responden preguntas como:

- ¿Para qué se creó este dataset?
- ¿Qué sesgos pueden existir?
- ¿Está permitido usarlo en producción?
- ¿Cómo se etiquetaron los datos?

```
# — Ejemplo de metadatos de un DataFrame de pandas —
from sklearn.datasets import load_iris
import json

iris_sk = load_iris(as_frame=True)
df_iris = iris_sk.frame

# Metadatos que podemos extraer programáticamente
metadatos = {
    "nombre": "Iris Dataset",
    "fuente": "sklearn.datasets.load_iris (original: UCI ML Repository)",
    "licencia": "Dominio público",
    "version": "1.0",
    "n_filas": df_iris.shape[0],
    "n_columnas": df_iris.shape[1],
    "columnas": {
        col: {
            "tipo_pandas": str(df_iris[col].dtype),
            "tipo_estadist": "numérica continua" if df_iris[col].dtype != 'object' else "categorial",
            "nulos_pct": round(df_iris[col].isna().mean() * 100, 2),
            "min": round(float(df_iris[col].min()), 3) if df_iris[col].dtype != 'object' else df_iris[col].min(),
            "max": round(float(df_iris[col].max()), 3) if df_iris[col].dtype != 'object' else df_iris[col].max(),
            "media": round(float(df_iris[col].mean()), 3) if df_iris[col].dtype != 'object' else df_iris[col].mean()
        }
        for col in df_iris.columns
    },
    "clases_target": iris_sk.target_names.tolist(),
    "balanceo_target": df_iris['target'].value_counts(normalize=True).round(3).to_dict(),
    "tarea_ml": "Clasificación multiclase (3 clases)",
    "uso_permitido": "Docencia, investigación, benchmarking"
}

print("Data Card del dataset Iris:")
print(json.dumps(metadatos, indent=2, ensure_ascii=False))
```

Data Card del dataset Iris:

```
{
  "nombre": "Iris Dataset",
  "fuente": "sklearn.datasets.load_iris (original: UCI ML Repository)",
  "licencia": "Dominio público",
  "version": "1.0",
  "n_filas": 150,
  "n_columnas": 5,
  "columnas": {
    "sepal length (cm)": {
      "tipo_pandas": "float64",
      "tipo_estadist": "numérica continua",
      "nulos_pct": 0.0,
      "min": 4.3,
      "max": 7.9,
      "media": 5.843
    },
    "sepal width (cm)": {
      "tipo_pandas": "float64",
      "tipo_estadist": "numérica continua",
      "nulos_pct": 0.0,
      "min": 2.0,
      "max": 4.4,
      "media": 3.057
    },
    "petal length (cm)": {
      "tipo_pandas": "float64",
      "tipo_estadist": "numérica continua",
      "nulos_pct": 0.0,
      "min": 1.0,
      "max": 6.9,
      "media": 3.758
    },
    "petal width (cm)": {
      "tipo_pandas": "float64",
      "tipo_estadist": "numérica continua",
      "nulos_pct": 0.0,
      "min": 0.1,
      "max": 2.5,
      "media": 1.199
    },
    "target": {
      "tipo_pandas": "int64",
      "tipo_estadist": "numérica continua",
      "nulos_pct": 0.0,
      "min": 0.0,
      "max": 2.0,
      "media": 1.0
    }
  },
  "clases_target": [
    "setosa",
    "versicolor",
    "virginica"
  ],
  "balanceo_target": {
    "0": 0.333,
    "1": 0.333,
    "2": 0.333
  },
}
```

```

"tarea_ml": "Clasificación multiclase (3 clases)",
"uso_permitido": "Docencia, investigación, benchmarking"
}

```

```

# — Metadatos en ficheros Parquet: el formato lleva los metadatos integrados —
import io

# Convertimos el df_iris a Parquet en memoria y leemos sus metadatos
try:
    import pyarrow.parquet as pq
    import pyarrow as pa

    buffer = io.BytesIO()
    df_iris.to_parquet(buffer, engine='pyarrow', index=False)
    buffer.seek(0)

    pf = pq.ParquetFile(buffer)
    meta = pf.metadata
    schema = pf.schema_arrow

    print("Metadatos integrados en un fichero Parquet:")
    print(f"  Filas totales:      {meta.num_rows}")
    print(f"  Grupos de filas:    {meta.num_row_groups}")
    print(f"  Columnas:           {meta.num_columns}")
    print(f"  Creado con:         {meta.created_by}")
    print(f"\nEsquema Arrow:")
    for field in schema:
        print(f"  {field.name:<25} {str(field.type)}")
    print("\n→ Parquet es el formato preferido en pipelines ML: comprimido, columna")
except ImportError:
    print("pyarrow no disponible en este entorno.")
    print("Instalación: pip install pyarrow")
    print("\nParquet almacena internamente: esquema, tipos, estadísticas por columna")
    print("metadatos de compresión y codificación → ideal para ML pipelines.")

```

pyarrow no disponible en este entorno.
 Instalación: pip install pyarrow

Parquet almacena internamente: esquema, tipos, estadísticas por columna,
 metadatos de compresión y codificación → ideal para ML pipelines.

1.7 Tendencias actuales en arquitecturas de datos para ML (2024-2026)

El ecosistema de datos para ML evoluciona rápidamente. Estas son las tendencias que están redefiniendo cómo se construyen los pipelines de datos en proyectos de IA.



Data Lakehouse

Combina lo mejor del Data Lake (almacenamiento barato, cualquier formato) y el Data Warehouse (transacciones ACID, SQL estándar). Formatos como **Delta Lake** (Databricks) y **Apache Iceberg** permiten hacer SQL y ML sobre los mismos datos sin duplicarlos.

Data Lake	+	Data Warehouse	=	Data Lakehouse
↓		↓		↓
Barato, flexible		SQL, ACID		Parquet/Iceberg + SQL + ML

Feature Stores

Un **feature store** es un registro centralizado de *features* de ML: garantiza que el modelo en producción usa exactamente las mismas transformaciones que durante el entrenamiento (evitando el *training-serving skew*). Herramientas clave: **Feast** (open source), **Hopsworks**, **Tecton**, **Vertex AI Feature Store**.

Data Mesh

Arquitectura que distribuye la propiedad de los datos entre los dominios de negocio (en lugar de un equipo central de datos). Cada dominio publica sus datos como "productos de datos" con metadatos, contratos y SLAs. Está ganando terreno en empresas grandes.

Datos para LLMs y RAG

Los modelos de lenguaje grandes (LLMs) han introducido nuevas necesidades:

- **Bases de datos vectoriales** (Pinecone, Weaviate, pgvector, Chroma) para almacenar y buscar embeddings.
- **RAG** (Retrieval-Augmented Generation): el pipeline extrae documentos relevantes de una fuente y los pasa como contexto al LLM.

- **Datos sintéticos:** generar datasets de entrenamiento con LLMs para reducir dependencia de datos etiquetados.

```
# — Comparativa de tecnologías actuales para datos en ML —————
print("Ecosistema de herramientas de datos para ML (2024-2026):")
print("=" * 70)

categorias_tech = {
    "[Almacenamiento / Formatos]": [
        ("Parquet / Arrow", "Columnar, comprimido, con esquema. Estándar de facto"),
        ("Delta Lake", "Parquet + ACID + time travel. Lakehouse open source."),
        ("Apache Iceberg", "Alternativa a Delta, más neutral (Netflix, Apple)."),
    ],
    "[Orquestación de pipelines]": [
        ("Apache Airflow", "DAGs de tareas, el más extendido. Ideal para batch."),
        ("Prefect / Dagster", "Modernos, más fáciles de testear. Python-first."),
        ("dbt", "Transformaciones SQL versionadas. DataOps."),
    ],
    "[Feature Stores]": [
        ("Feast", "Open source. Integrado con Redis, BigQuery, Snowflake"),
        ("Hopsworks", "Feature store + Jupyter + MLOps integrado."),
        ("Tecton", "Enterprise, muy maduro, usado en Uber/Twitter."),
    ],
    "[Vector Databases – para LLMs/RAG]": [
        ("pgvector", "Extension PostgreSQL. Fácil si ya usas SQL."),
        ("Chroma", "Open source, muy usado en prototipos RAG."),
        ("Pinecone / Weaviate", "Managed, producción, búsqueda semántica a escala."),
    ],
    "[Observabilidad y calidad]": [
        ("Great Expectations", "Validación de datos con 'expectations'. DQ as code."),
        ("deequ (AWS)", "Librería Spark para métricas de calidad a escala."),
        ("MLflow", "Trazabilidad de experimentos, modelos y datasets."),
    ],
}

for categoria, techs in categorias_tech.items():
    print(f"\n{categoria}")
    print("-" * 60)
    for nombre, descripcion in techs:
        print(f"  {nombre:<22} {descripcion}")
```

Ecosistema de herramientas de datos para ML (2024-2026):
=====

[Almacenamiento / Formatos]

Parquet / Arrow	Columnar, comprimido, con esquema. Estándar de facto en ML.
Delta Lake	Parquet + ACID + time travel. Lakehouse open source.
Apache Iceberg	Alternativa a Delta, más neutral (Netflix, Apple).

[Orquestacion de pipelines]

Apache Airflow	DAGs de tareas, el más extendido. Ideal para batch.
Prefect / Dagster	Modernos, más fáciles de testear. Python-first.
dbt	Transformaciones SQL versionadas. DataOps.

[Feature Stores]

Feast	Open source. Integrado con Redis, BigQuery, Snowflake.
Hopsworks	Feature store + Jupyter + MLOps integrado.
Tecton	Enterprise, muy maduro, usado en Uber/Twitter.

[Vector Databases – para LLMs/RAG]

pgvector	Extension PostgreSQL. Facil si ya usas SQL.
Chroma	Open source, muy usado en prototipos RAG.
Pinecone / Weaviate	Managed, produccion, busqueda semantica a escala.

[Observabilidad y calidad]

Great Expectations	Validacion de datos con 'expectations'. DQ as code.
deequ (AWS)	Libreria Spark para metricas de calidad a escala.
MLflow	Trazabilidad de experimentos, modelos y datasets.

Resumen del Bloque 1

Concepto	Idea clave
Datos estructurados	Tabular, esquema fijo. Formato nativo de pandas y scikit-learn.
Datos semiestructurados	JSON, XML. Hay que aplanarlos antes de entrenar.
Datos no estructurados	Texto, imagen, audio. Requieren embeddings o deep learning.
Clasificación por origen	SQL, NoSQL, APIs, streaming, IoT, repositorios públicos.
Fuentes formales vs no formales	Las formales tienen mejor calidad; las no formales, más volumen.
Ubicación	Local (control), nube (escala), híbrido (equilibrio). Afecta a coste y privacidad.
Repositorios públicos	Kaggle, HuggingFace, UCI, OpenML — esenciales para prototipado y benchmarking.
Metadatos	Describen qué son los datos. Críticos para reproducibilidad y gobernanza.
Data Lakehouse	Unifica Data Lake + Data Warehouse. Delta Lake, Apache Iceberg.
Feature Store	Garantiza consistencia entre entrenamiento e inferencia.
Bases vectoriales	Nuevo paradigma para RAG y búsqueda semántica con LLMs.

Siguiente bloque

Bloque 2 — Principales sistemas de almacenamiento y formatos: profundizaremos en SQL, NoSQL, y los formatos de fichero orientados a ML (Parquet, Arrow, HDF5), con ejemplos prácticos de extracción y escritura con Python.